

LABORATORY OBSERVATION

Course	Full Stack Web Development
Course Code	23CM4121
Observation Task No.	3
Student Name	M. Saicharan
Roll No.	A24126552096
Date	30-08-2026

1. TITLE

Design and Implementation of a Student Information Generator Using JavaScript Classes and DOM Manipulation

2. AIM

To develop an interactive Student Information Generator using HTML, CSS, and JavaScript that accepts student details and dynamically creates a formatted profile card using a JavaScript class and DOM manipulation.

3. OBJECTIVE

1. To create a student input form using HTML.
2. To style the form and generated profile using CSS.
3. To define a JavaScript class with a constructor for student data.
4. To create a student object from user-entered values.
5. To handle button click events using JavaScript.
6. To dynamically create HTML elements using createElement().
7. To display student information by manipulating the DOM.
8. To use CSS Grid for aligned profile information.

4. THEORY

JavaScript classes provide a blueprint for creating objects with properties and methods. A constructor initializes the properties of an object. The Document Object Model (DOM) allows JavaScript to access and modify HTML elements dynamically. In this program, user input is read using getElementById(). A StudentDetails object is created using the entered values. The program then creates a profile card dynamically using createElement(), assigns CSS classes, inserts the student data using innerHTML, and adds the generated card to the webpage using appendChild().

Application Flow: User Input → Create Student Object → Button Event → DOM Element Creation → Display Profile Card

5. ALGORITHM / PROCEDURE

1. Create an HTML5 document titled 'Student Information Generator'.
2. Design a centered input box using CSS.
3. Create input fields for name, roll number, department, and CGPA.
4. Create a Create Profile button.
5. Define the StudentDetails JavaScript class and constructor.
6. Read the values entered by the user using getElementById().
7. Create a StudentDetails object using the entered values.
8. Clear any previously generated profile content.

9. Create a new div element dynamically for the profile.
10. Create a heading element and a details container.
11. Insert student information into the details container.
12. Append the heading and details to the profile card.
13. Append the completed profile card to the webpage.
14. Run the program in a browser and verify the generated profile.

6. PROGRAM

profile.html

```
<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="UTF-8">
  <title>Student Information Generator</title>

  <style>

    body {
      font-family: Arial, sans-serif;
      background: #e8f0fe;
      display: flex;
      justify-content: center;
      align-items: center;
      min-height: 100vh;
      margin: 0;
    }

    .input-box {
      background: white;
      padding: 30px;
      border-radius: 15px;
      box-shadow: 0 6px 15px rgba(0,0,0,0.15);
      text-align: center;
      width: 380px;
    }

    h2 {
      margin-top: 0;
      font-size: 26px;
      color: #222;
    }

    input,
    button {
      display: block;
      margin: 12px auto;
      padding: 12px;
      width: 90%;
      font-size: 16px;
      border-radius: 6px;
      box-sizing: border-box;
    }

    input {
      border: 1px solid #bbb;
    }

    button {
      background: #673ab7;
      color: white;
      border: none;
      font-weight: bold;
      cursor: pointer;
    }

    button:hover {
      background: #4527a0;
    }

    .profile {
      background: #263238;
      color: white;
      padding: 25px;
      margin-top: 20px;
      border-radius: 10px;
      box-shadow: 0 5px 12px rgba(0,0,0,0.2);
    }

    .profile h3 {
      margin-top: 0;
      border-bottom: 2px solid #ff9800;
      padding-bottom: 8px;
      font-size: 22px;
      color: #ff9800;
    }

    .details {
      display: grid;
      grid-template-columns: 110px auto;
      row-gap: 10px;
      text-align: left;
      font-size: 17px;
    }

  </style>
</head>

<body>

  <div class="input-box">
```

```

<h2>Student Information</h2>

<input type="text" id="studentName" placeholder="Enter Name">

<input type="text" id="studentRoll" placeholder="Enter Roll Number">

<input type="text" id="studentDept" placeholder="Enter Department">

<input type="number" step="0.01" id="studentCgpa" placeholder="Enter CGPA">

<button id="generateBtn">
  Create Profile
</button>

<div id="profileContainer"></div>

</div>

<script>

  class StudentDetails {

    constructor(name, roll, department, cgpa) {
      this.name = name;
      this.roll = roll;
      this.department = department;
      this.cgpa = cgpa;
    }

  }

  document.getElementById("generateBtn").onclick = function() {

    const student = new StudentDetails(

      document.getElementById("studentName").value,
      document.getElementById("studentRoll").value,
      document.getElementById("studentDept").value,
      document.getElementById("studentCgpa").value

    );

    const output = document.getElementById("profileContainer");

    output.innerHTML = "";

    const profileCard = document.createElement("div");

    profileCard.className = "profile";

    const heading = document.createElement("h3");

    heading.innerText = "Student Information";

    const details = document.createElement("div");

    details.className = "details";

    details.innerHTML = `

      <span>Name</span>
      <span>: ${student.name}</span>

      <span>Roll No</span>
      <span>: ${student.roll}</span>

      <span>Department</span>
      <span>: ${student.department}</span>

      <span>CGPA</span>
      <span>: ${student.cgpa}</span>

    `;

    profileCard.appendChild(heading);

    profileCard.appendChild(details);

    output.appendChild(profileCard);

  };

</script>

</body>

</html>

```

7. CODE EXPLANATION

Code / Syntax	Explanation
class StudentDetails	Defines a class used to represent student information.
constructor()	Initializes name, roll number, department, and CGPA properties.
getElementById()	Selects HTML elements using their unique IDs.
onclick	Executes the profile generation code when the button is clicked.
createElement()	Creates a new HTML element dynamically using JavaScript.
className	Assigns a CSS class to the dynamically created profile card.
innerHTML	Inserts student information into the details container.
appendChild()	Adds dynamically created elements to the webpage DOM.
CSS Grid	Aligns profile labels and values into two columns.

8. TEST CASES AND EXECUTION DATA

Test Case	Input / Action	Expected Result	Actual Result	Status
TC-01	Enter name	Name is accepted in the input field	Accepted	Pass
TC-02	Enter roll number	Roll number is accepted	Accepted	Pass
TC-03	Enter department and CGPA	Details are accepted	Accepted	Pass
TC-04	Click Create Profile	Student profile card is generated	Generated correctly	Pass
TC-05	Generate profile again	Previous profile is cleared and replaced	Updated correctly	Pass

9. OUTPUT

Output 1: Student Information form containing fields for Name, Roll Number, Department, and CGPA.

Output 2: After clicking Create Profile, a formatted Student Information card is dynamically generated displaying the entered student details.

Student Information
Name: B.JATIN Roll No: A24126552070 Department: CSE CGPA: 8.50

10. RESULT

Thus, the Student Information Generator was successfully designed and implemented using HTML, CSS, and JavaScript. The program successfully accepted student details, created a JavaScript object, and dynamically generated a formatted student profile card using DOM manipulation.